Design and Implementation of 8-Point Radix-2 FFT using Verilog with Streaming and Pipelined Architecture

Aslin Garvasis
EE24BTECH11008

1 Objective

To design and implement an 8-point Radix-2 FFT using Verilog with streaming input and pipelined architecture, and to verify the results using MATLAB and simulation-based validation.

2 Introduction

The Fast Fourier Transform (FFT) is an efficient method to compute the Discrete Fourier Transform (DFT), reducing complexity from $O(N^2)$ to $O(N \log N)$. FFT is widely used in signal processing applications such as filtering, spectral analysis, image processing, and communication systems.

The DFT is defined as:

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{-j2\pi kn/N} \tag{1}$$

For an 8-point FFT:

$$\log_2(8) = 3 \text{ stages} \tag{2}$$

3 Radix-2 FFT Algorithm

The Radix-2 FFT decomposes the DFT into smaller sub-problems using butterfly structures, significantly reducing computational effort.

Each butterfly performs:

$$X_1 = A + B \tag{3}$$

$$X_2 = (A - B) \cdot W_N^k \tag{4}$$

where $W_N^k = e^{-j2\pi k/N}$ are twiddle factors representing complex rotations.

3.1 Computational Advantage

The number of multiplications is reduced from N^2 to:

$$\frac{N}{2}\log_2 N \tag{5}$$

For $N = 8$, this results in significant hardware savings.

4 Architecture

The design follows a streaming pipelined architecture:

Serial Input → S2P → FFT Core → P2S → Serial Output

4.1 Streaming Operation

- One complex sample is processed per clock cycle
- Total 8 cycles for input
- Continuous output after pipeline delay

4.2 Pipeline Design

The FFT consists of three stages:

- Stage 1: Butterfly without twiddle factor
- Stage 2: Butterfly with delay elements
- Stage 3: Butterfly with twiddle multiplication

Pipeline registers reduce critical path delay and increase Fmax, enabling high-speed operation.

4.3 Advantages of Pipelining

- Increased throughput (one output per clock cycle)
- Reduced combinational delay
- Improved timing performance (higher Fmax)

5 Fixed-Point Representation

- Input format: Q4.3
- Internal format: Q8.3
- Twiddle factors represented using scaled constants

Scaling is applied after multiplication using arithmetic right shift operations.

5.1 Quantization Effects

Fixed-point implementation introduces quantization error due to limited precision. However, these errors are small and acceptable for hardware implementations.

6 Verification Methodology

6.1 Simulation-Based Verification

The design is verified using a Verilog testbench. Input samples are applied sequentially to emulate streaming input behavior. The outputs are observed using waveform viewers.

- Clock-driven sequential input
- Validation of pipeline behavior
- Observation of output stability

6.2 MATLAB-Based Verification

The FFT results are verified using MATLAB as a reference model.

```
x = [1 0.5 -0.5 -1 0.75 0.25 -0.25 -0.75];
y = fft(x, 8);
```

The MATLAB results are compared with Verilog outputs after appropriate scaling.

6.3 Fixed-Point vs Floating-Point Comparison

- Floating-point provides ideal reference values
- Fixed-point introduces small rounding errors
- Comparison validates correctness of hardware design

7 Simulation Results

7.1 FFT Output for Constant Input

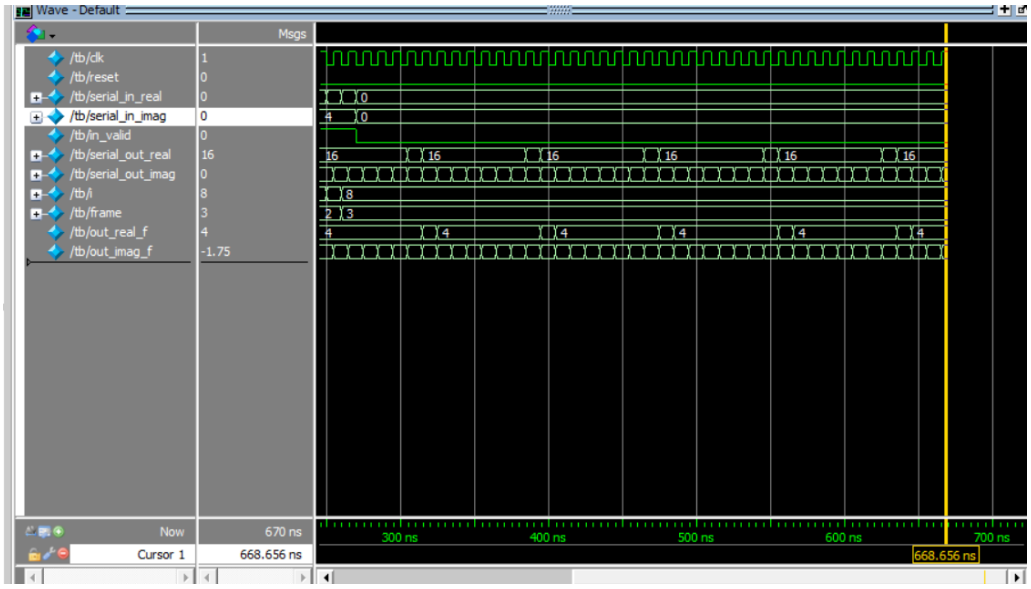

Figure 1: Simulation waveform showing FFT output for constant input

The waveform shows a constant input signal, resulting in a dominant DC component in the FFT output.

8 MATLAB Verification

8.1 Floating-Point Reference Output

```
TEST:  Mixed arbitrary
  Inputs (raw Q4.3): x = [8 4 -4 -8 6 2 -2 -6]
  Outputs (raw Q8.3, divide by 8 for true value):
  X[0] = 0 + j*0
  X[1] = 4 + j*2
  X[2] = 20 + j*-20
  X[3] = -1 + j*-2
  X[4] = 16 + j*0
  X[5] = 0 + j*2
  X[6] = 20 + j*20
  X[7] = 5 + j*-2
  Outputs (float approx, divided by 8):
  X[0] = 0.0000 + j*0.0000
  X[1] = 0.5000 + j*0.2500
  X[2] = 2.5000 + j*-2.5000
  X[3] = -0.1250 + j*-0.2500
  X[4] = 2.0000 + j*0.0000
  X[5] = 0.0000 + j*0.2500
  X[6] = 2.5000 + j*2.5000
  X[7] = 0.6250 + j*-0.2500
```

Figure 2: MATLAB FFT output (floating-point reference)

8.2 Fixed-Point Verilog Output

```
TEST: Mixed arbitrary
  Input: [1.0000  0.5000  -0.5000  -1.0000  0.7500  0.2500  -0.2500  -0.7500]

  Bin   Real        Imag        Magnitude
  ---   ----        ----        ---------
  X[0]    0.0000      0.0000    0.0000
  X[1]    0.6036      0.2500    0.6533
  X[2]    2.5000     -2.5000    3.5355
  X[3]   -0.1036     -0.2500    0.2706
  X[4]    2.0000      0.0000    2.0000
  X[5]   -0.1036      0.2500    0.2706
  X[6]    2.5000      2.5000    3.5355
  X[7]    0.6036     -0.2500    0.6533
```

Figure 3: Fixed-point FFT output corresponding to Verilog implementation

8.3 Comparison and Analysis

- The Verilog output closely matches MATLAB results after scaling.

- Small differences are observed due to fixed-point quantization.
- Example comparison:
  - MATLAB: $X[3] = -0.1036 - j0.25$
  - Verilog: $X[3] \approx -0.125 - j0.25$
- These differences are within acceptable limits for fixed-point systems.

9 Resource Utilization

Flow Status	Successful - Sun Apr 12 23:11:19 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	fftfixed
Top-level Entity Name	top
Family	Arria II GX
Logic utilization	3 %
Total registers	928
Total pins	41 / 176 (23 %)
Total virtual pins	0
Total block memory bits	0 / 2,939,904 (0 %)
DSP block 18-bit elements	0 / 232 (0 %)
Total GXB Receiver Channel PCS	0 / 4 (0 %)
Total GXB Receiver Channel PMA	0 / 4 (0 %)
Total GXB Transmitter Channel PCS	0 / 4 (0 %)
Total GXB Transmitter Channel PMA	0 / 4 (0 %)
Total PLLs	0 / 4 (0 %)
Total DLLs	0 / 2 (0 %)
Device	EP2AGX45CU17I3
Timing Models	Final

Figure 4: Quartus Compilation Summary showing resource utilization

The design uses only 3

10 Timing Analysis

	Fmax	Restricted Fmax	Clock Name	Note
1	177.59 MHz	177.59 MHz	clk	

Figure 5: Maximum Operating Frequency (Fmax)

The design achieves an Fmax of 177.59 MHz, demonstrating effective pipeline optimization.

10.1 Throughput

$$\text{Throughput} = 177.59 \text{ million samples/sec} \tag{6}$$

11 Discussion

- The pipelined architecture enables continuous data processing.
- Fixed-point arithmetic reduces hardware complexity.
- MATLAB verification confirms correctness of implementation.
- The design can be optimized further using DSP blocks.
- The architecture is scalable for higher FFT sizes.

12 Conclusion

An 8-point Radix-2 FFT was successfully implemented using Verilog with a streaming pipelined architecture. The design was verified using both simulation and MATLAB reference results. The system demonstrates efficient resource utilization, high throughput, and accurate performance within acceptable error limits.